Week 06 — Practical: From Communication to Messaging

> Slides: week06-iot-edge-06.pdf Theory behind it: week 05 notes (IP, TCP/UDP, MQTT).

Goal: observe how **low-level transport communication becomes application-level IoT messaging and action**.

```
Python network programming  ->  MQTT  ->  Edge action
TCP sockets | UDP sockets      MQTT messaging    Edge logic | Virtual actuator
```

Practical flow

Activity	Main output
Networking basics	IP, port, socket, TCP/UDP socket types
TCP socket programming	Client/server exchange JSON
UDP socket programming	Datagram send/receive + receiver-off experiment
TCP vs UDP interpretation	Observation table
Transition to MQTT	Why application protocols are needed
MQTT publisher/subscriber	Brokered sensor data messaging
Edge processor + actuator	MQTT command generated from sensor data
Node-RED demo + submission	Visual flow + evidence collection

Setup

```
network-mqtt/
├── tcp_server.py
├── tcp_client.py
├── udp_receiver.py
├── udp_sender.py
├── mqtt_subscriber.py
├── mqtt_sensor.py
├── edge_processor.py
└── mqtt_actuator.py
```

Required software: Python 3.x, `pip install paho-mqtt`, the **Mosquitto** broker installed and running, and **Node-RED** for the final visual flow. Use **127.0.0.1** for everything, and **separate terminals** for server, client, broker, sensor and actuator.

Networking basics

Three concepts

Concept	Meaning in this practical	Example
IP address	Identifies the **host**	`127.0.0.1` = localhost
Port	Identifies the **application/process**	5000, 5001, **1883**
Socket	**Programming interface** to transport communication	TCP or UDP socket

```
HOST = "127.0.0.1"   # loopback - this same computer
PORT = 5000          # application endpoint
```

- `127.0.0.1:1883` → connect to an MQTT broker **on the same PC**.
- A **LAN IP** identifies a device on the local network: `192.168.0.25:1883` → the broker running on the computer whose LAN IP is 192.168.0.25.

Python socket types: TCP vs UDP

```
import socket

tcp_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)   # IPv4 + TCP
udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)    # IPv4 + UDP
```

`socket.socket(address_family, socket_type)` answers two questions: **what kind of network address** (IPv4 or IPv6) and **what kind of transport** (TCP or UDP).

- **AF** = Address Family, **INET** = internet protocol family → **AF_INET = IPv4** (use `AF_INET6` for IPv6).
- **SOCK_STREAM = TCP**, **SOCK_DGRAM = UDP**.

Python argument	Networking concept	Layer
`AF_INET`	IPv4	**Network** layer
`SOCK_STREAM`	TCP	**Transport** layer
`SOCK_DGRAM`	UDP	**Transport** layer

Stream vs datagram — the key difference:

TCP = STREAM	**UDP = DATAGRAM**
Sends a **continuous byte stream**	Each `sendto()` creates a **separate datagram**
Does not preserve boundaries of individual `send()` calls	**Message boundaries are preserved**
Send `"Hello"` then `"World"` → received as `"HelloWorld"`	Send `"Message A"` then `"Message B"` → datagram 1 and datagram 2

UDP does not guarantee that a packet arrives, that packets arrive **in order**, or that packets are **not duplicated**.

> **Simplified analogy: TCP = sealed courier delivery** (trackable, missing data resent); **UDP = postcard** (lightweight, connectionless).

Where the socket sits: temperature sensor → IoT device → Python program → **socket** → network → server.

TCP client, three steps:

```
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  # 1. create
client.connect(("127.0.0.1", 5000))                         # 2. connect
client.send(b"Hello")                                       # 3. send bytes
client.close()
```

`127.0.0.1` = destination IP, `5000` = destination port. **A socket transmits bytes, not strings.**

UDP client — no connect:

```
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.sendto(b"Hello", ("127.0.0.1", 5000))
client.close()
```

Part 1 — TCP client/server

Goal: (1) establish connection, (2) send JSON, (3) receive response.

```
TCP client --connect()--> TCP server --recv()--> send response
```

- The server waits using **`listen()`** and **`accept()`**.
- The client **must** **`connect()`** before sending data.
- The payload is **JSON encoded as bytes**.
- The server receives (`recv()`) and sends an **application-level response**.

`tcp_server.py`

Call	What it does
`socket.AF_INET`	IPv4 addressing
`SOCK_STREAM`	Creates a TCP socket
`bind()`	**Reserves the server endpoint**
`listen()`	Makes the program **wait for incoming TCP clients**
`accept()`	**Blocks until a client connects**
`recv(1024)`	Receives up to 1024 bytes
`decode()`	Bytes → text
`json.loads()`	JSON text → Python dictionary
`sendall()`	Sends the application response back

`tcp_client.py`

Call	What it does
`sensor_data`	The application data
`json.dumps()`	Serializes the dictionary into a **JSON string**
`connect()`	**Practical evidence that TCP is connection-oriented** — if no server is listening, it **fails**
`encode()`	Text → bytes
`sendall()`	Sends bytes over the connection
`recv()`	Waits for response bytes
`decode()`	Response bytes → text

Run order (different terminals): `python tcp_server.py`, then `python tcp_client.py`.

TCP experiment: stop the server

> **Steps:** stop `tcp_server.py` → run `tcp_client.py` again → observe the connection error. **Question:** what happens if the client runs **before** the server is listening?

Expected explanation: TCP is connection-oriented. The client **cannot exchange application data unless a listening server accepts the connection**.

Part 2 — UDP sender/receiver

Goal: send **the same JSON data** as a UDP datagram.

```
UDP sender --sendto()--> datagram --recvfrom()--> UDP receiver
```

- **No** `connect()` call is needed.
- The sender sends a datagram **to an IP + port**.
- The receiver gets one datagram using `recvfrom()`, which returns **data and sender address**.
- **The sender may not know whether an application received the message.**
- There is **no** `accept()`, because UDP has **no connection setup**.

> **UDP still uses IP addresses and ports. The difference is delivery behavior, not addressing.**

`udp_receiver.py` binds to a UDP port, receives JSON-encoded sensor data as **raw bytes**, decodes it to a string, then parses it into a dictionary. `udp_sender.py` just calls `sendto()` with the destination endpoint — **"message sent" does not prove the receiving application obtained it.**

Run order: `python udp_receiver.py`, then `python udp_sender.py`.

UDP experiment: stop the receiver

> **Steps:** run both normally → stop `udp_receiver.py` → run `udp_sender.py` again → observe that the sender **may still print "UDP message sent"**. **Questions:** did the receiver application receive anything? Does the sender automatically know?

Expected explanation: No. UDP **does not establish a connection** and **does not provide built-in delivery confirmation**.

TCP vs UDP: practical observations

Observation	TCP	UDP
Connection setup	**Required**	**Not required**
Main send function	`sendall()`	`sendto()`
Main receive function	`recv()`	`recvfrom()`
Uses IP + port	Yes	Yes
Built-in reliability/order	Yes	**No built-in guarantee**
Data model	**Byte stream**	**Datagram**

> **Key lesson: the payload can be the same JSON data; the *transport behavior* changes.**

Same IoT data, different transport: `Python dict → JSON string → bytes → TCP or UDP`. The **sensor meaning** did not change, the **JSON representation** did not change — only the **transport mechanism** changed.

From sockets to application protocols

```
Application data -> Application protocol -> TCP/UDP -> IP -> Connectivity
```

Raw socket programming gives	MQTT adds IoT messaging concepts
Direct control over TCP/UDP communication	**Publisher and subscriber roles**
IP + port endpoints	**Broker-based message distribution**
Bytes sent and received by applications	**Topics and wildcard subscriptions**
	QoS and retained messages

TCP/UDP only move bytes — they do not define "publish", "subscribe", "topic" or "device command".

> **MQTT typically runs over TCP/IP. MQTT does not replace TCP; it uses TCP and adds application-level messaging.**

Part 3 — MQTT publisher/subscriber

```
Python sensor (publisher) -> MQTT broker -> Python subscriber or Node-RED
```

The publisher knows	The publisher does **not** need to know
Broker address	Subscriber IP address
Topic	How many subscribers exist
Payload	Whether a dashboard, database or Node-RED receives the message
QoS / retained options	

> **The architectural benefit: producer and consumer are decoupled through the broker.**

Mosquitto broker setup

1. Install Mosquitto (https://mosquitto.org/download/; default `C:\Program Files\mosquitto`).
2. Add it to the Windows **Path**: Windows key → *Environment Variables* → *Edit the system environment variables* → **Environment Variables...** → under *User variables*, select **Path** → **Edit** → **New** → add `C:\Program Files\mosquitto` → OK. Reopen VS Code and test with `mosquitto -v`.
3. Test the broker with three terminals:

```
mosquitto -v
```

```
mosquitto_sub -h localhost -t "test/topic" -v
```

```
mosquitto_pub -h localhost -t "test/topic" -m "Hello IoT"
```

Expected output in the subscriber: `test/topic Hello IoT`

Part	Meaning
`-v`	Prints connection and message activity
`mosquitto_sub`	MQTT subscriber client
`mosquitto_pub`	MQTT publisher client
`-h localhost`	Connect to the local Mosquitto broker
`-t "test/topic"`	Publish/subscribe to this topic
`-m "Hello IoT"`	Message payload

> **MQTT routes messages through the broker; clients do not talk directly to each other.**

MQTT default port: 1883. For all scripts: `BROKER = "127.0.0.1"`, `PORT = 1883`.

Terminal plan: 1 — `mosquitto -v`; 2 — subscriber; 3 — sensor publisher; 4 — edge processor / actuator.

`mqtt_subscriber.py`

1. **MQTT configuration** — `BROKER` (where is the broker?), `PORT` (which broker service?), `TOPIC` (which messages do we want?).
2. **Connection callback** — `on_connect()` runs when the connection event occurs; `reason_code` says whether it succeeded; then `client.subscribe(TOPIC)` tells the broker "*send me messages published to this topic*".
3. **Message callback** — `on_message()` is called automatically whenever a matching message arrives; `message.topic` is the MQTT topic and `message.payload` the actual application data.
4. **Create and configure the client** — `VERSION2` uses Paho's current callback interface; connection → `on_connect()`, message → `on_message()`.
5. **Connect and wait** — keep the client running so it can process network traffic.

```
Connect -> Subscribe -> Wait -> Message arrives -> on_message() -> Wait again
```

> **Run `mqtt_subscriber.py` before `mqtt_sensor.py`.**

`mqtt_sensor.py` (simulated sensor publisher)

- The client connects to the local broker; `TOPIC` **classifies** the message.
- `loop_start()` runs MQTT network processing **in the background**.
- `keepalive = 60` tells the broker: *if there is no normal MQTT traffic for up to 60 seconds, I will still send a keep-alive message so the connection stays active.*
- `random.uniform()` simulates a changing sensor reading; `json.dumps()` serializes it into a **JSON payload**; `client.publish()` sends it to the broker.
- `qos=0` means **no MQTT-level acknowledgment or redelivery**. `Ctrl+C` stops the infinite loop.
- **The publisher still does not know subscriber addresses.**

Experiment: multiple subscribers

> Open **two** terminals running `mqtt_subscriber.py`, run `mqtt_sensor.py` once, and observe that **both subscribers receive the same publication — with no change to the publisher code**.

Expected explanation: the publisher sends to the **broker and topic**; the **broker distributes the message to all matching subscribers**. This is producer–consumer decoupling in practice.

Experiment: topic wildcards

Copy `mqtt_sensor.py` to `mqtt_sensor_wildcard.py`, modify it to publish to **several topics** per loop, then test the subscriber with `+` and `#`:

Wildcard	Meaning	Example
`+`	Exactly **one** topic level	`factory/+/temperature`
`#`	**All remaining** levels	`factory/#`

Part 4 — Python edge processor

```
Sensor publisher -> MQTT broker -> Edge processor -> MQTT command -> Actuator subscriber
```

Edge processing task:

- Subscribe to `factory/machine01/temperature`
- Parse the JSON payload
- If **temperature > 30 °C**, publish **FAN ON**; otherwise publish **FAN OFF**
- Command topic: `factory/machine01/fan/command`

`edge_processor.py`

> The edge processor is an **MQTT subscriber and publisher at the same time**.

- `SENSOR_TOPIC` receives temperature messages; `COMMAND_TOPIC` sends fan commands.
- `on_connect()` subscribes **only after the broker connection succeeds**.
- `json.loads()` parses the payload into a dictionary. **The temperature is application data, not MQTT metadata.**
- The `if/else` is the **edge decision rule**; `client.publish()` sends the command to another topic.

> **This creates a simple sense–decide–act loop.**

`mqtt_actuator.py` (virtual actuator)

- The actuator is an **MQTT subscriber**, and it subscribes to the **command topic, not the sensor topic**.
- `message.payload` is **bytes**; `decode()` converts the command to text.
- The command is **application-defined**: `FAN ON` / `FAN OFF` simulates physical actuation, which **closes the control loop**.
- **Real hardware could replace the print statements later.**

Running the complete loop

Terminal	Command
1	`mosquitto -v`
2	`python mqtt_sensor.py`
3	`python edge_processor.py`
4	`python mqtt_actuator.py`

Observation: when the random temperature rises above **30 °C**, the command should become **FAN ON**.

Interpreting the edge loop

Component	Role in the IoT system
`mqtt_sensor.py`	Simulated sensing device / publisher
JSON payload	Structured data representation
Mosquitto	Local MQTT broker / edge communication point
`edge_processor.py`	**Local processing and decision logic**
`mqtt_actuator.py`	Virtual actuator / subscriber
`FAN ON` / `FAN OFF`	Application-defined command

> **Complete simulated IoT loop: sense → communicate → process → act.**

Part 5 — Node-RED visual flow demo

```
MQTT In -> JSON -> Switch -> FAN ON/OFF -> MQTT Out
```

Node-RED shows MQTT message input, JSON parsing, the temperature threshold decision, the virtual action command, and MQTT output to the actuator topic.

> **Node-RED is not mainly a network simulator** — here it is a **visual IoT integration and application-logic tool**.

Install

1. Install **Node.js LTS** (https://nodejs.org/en/download).
2. Reopen VS Code and verify: `node --version`, `npm --version`.
3. `npm install -g --unsafe-perm node-red`
4. Run `node-red` — it starts at **http://127.0.0.1:1880**. (If `node-red` is not recognized, run `npm config get prefix`.)

Required nodes

Node	Purpose	Important setting
MQTT In	Subscribe to sensor data	topic: `factory/machine01/temperature`
JSON	Parse JSON text into an object	`msg.payload` becomes an object
Debug	Show received payload	complete `msg.payload`
Switch	Test the temperature threshold	`msg.payload.temperature > 30`
Change	Create the command	`msg.payload` = `FAN ON` / `FAN OFF`
MQTT Out	Publish the command	topic: `factory/machine01/fan/command`

Field	Meaning	Example
`msg.topic`	Where/what **category** the message belongs to	`factory/machine01/temperature`
`msg.payload`	The actual **data** carried by the message	`{"temperature":34.4,"unit":"C"}`

> `msg.topic` **= label/route;** `msg.payload` **= content.** (The same topic-vs-payload distinction as in week 5.)

Build order: (1) MQTT In → (2) JSON → (3) Debug → (4) Switch → (5) two Change nodes (FAN ON / FAN OFF) → (6) two Debug nodes → (7) MQTT Out (**topic: leave empty, QoS 0, retain false, name "Fan command"**).

Terminals for the demo: 1 — `mosquitto -v`; 2 — `python mqtt_sensor.py`; browser — Node-RED; 3 — `python mqtt_actuator.py`.

Summary

```
Python dict -> JSON -> TCP/UDP sockets -> MQTT -> Edge logic -> Action
```

Practical part	What it demonstrates
TCP sockets	Connection-oriented **process-to-process** communication
UDP sockets	**Connectionless** datagram communication
MQTT	**Brokered publish-subscribe** IoT messaging
Edge processor / Node-RED	**Local application logic close to the data source**
Virtual actuator	The **command/action** side of an IoT loop

Most important for exams

- **IP = host, port = application/process, socket = the programming interface** to transport communication. `127.0.0.1` = localhost; **1883 = MQTT**.
- **`socket.socket(AF_INET, SOCK_STREAM)` = IPv4 + TCP; `SOCK_DGRAM` = UDP.** Know that **AF_INET → network layer**, **SOCK_STREAM/SOCK_DGRAM → transport layer**.
- **TCP = byte stream (boundaries not preserved — "Hello"+"World" arrives as "HelloWorld"); UDP = datagrams (boundaries preserved).**
- **TCP calls:** `bind()` → `listen()` → `accept()` (blocks) → `recv()` → `sendall()`; client: `connect()` → `sendall()` → `recv()`. **UDP calls:** `bind()` → `recvfrom()` (returns data **and sender address**); sender: `sendto()` — **no `connect()`, no `accept()`**.
- **Sockets transmit bytes:** `json.dumps()` → `encode()` → send; receive → `decode()` → `json.loads()`.
- **The two experiments and their explanations:** stopping the TCP server makes `connect()` **fail** (TCP is connection-oriented); stopping the UDP receiver still prints "message sent" (**no connection, no delivery confirmation**).
- **TCP vs UDP observation table** — connection setup, `sendall()` / `sendto()`, `recv()` / `recvfrom()`, reliability, byte stream vs datagram.
- **Same JSON payload, different transport behavior** — the meaning and representation don't change, only the transport.
- **Why MQTT:** TCP/UDP only move bytes; MQTT adds **publisher/subscriber roles, a broker, topics and wildcards, QoS and retained messages**. **MQTT uses TCP, it does not replace it.**
- **Paho client structure:** `on_connect()` (then `subscribe()`), `on_message()` (`message.topic`, `message.payload`), `loop_start()`, `keepalive = 60`, `qos=0`.
- **Run the subscriber before the publisher**, and **`mosquitto -v` before everything**.
- **The multiple-subscriber experiment proves decoupling:** the publisher is unchanged, and the broker distributes to all matching subscribers.
- **The edge loop: sense → communicate → process → act**, with the edge processor acting as **both subscriber and publisher**, and the actuator subscribing to the **command topic**, not the sensor topic.
- **Node-RED:** MQTT In → JSON → Switch (`msg.payload.temperature > 30`) → Change (`FAN ON` / `FAN OFF`) → MQTT Out; **`msg.topic` = route, `msg.payload` = content**.